

Distil - Project Report

An LLM Governance & Token-Optimization Gateway

Live product: <https://distil.vercel.app>

MCP connector: <https://distil.vercel.app/mcp>

Source code: <https://github.com/ashritkvs/distil>

Date: July 2026

1. Executive summary

Distil is a finished, working product that is live on the internet right now. In simple terms, it is a **checkpoint that sits in front of an AI model**. **Every time someone (a person or a software "agent") wants to send a prompt** to an AI like ChatGPT or Claude, the prompt first passes through this checkpoint. At the checkpoint, the system does five things automatically:

1. **Checks the prompt is safe** - is it trying to trick the AI, leaking private data, asking for dangerous code, or requesting harmful content?
2. **Makes the prompt smaller** - it removes wasted words so the prompt uses fewer "tokens" (the unit AI companies charge money for).
3. **Confirms the meaning was kept** - it double-checks the shorter prompt still says the same thing.
4. **Measures everything** - how many tokens were saved, how much money, how much energy and carbon, how fast it ran.
5. **Tracks usage** - who used it, how much of their plan they have left.

This was built to satisfy **two source documents at the same time**: an **Observability Whitepaper** (a design for token-based monitoring) and the **TokenOps** brief (a list of features for a token-usage and safety product). The finished product covers **every feature in the TokenOps brief** and about **70% of the Observability Whitepaper** - the part that does not need special GPU hardware to build.

The product can be used three ways: as a **web page** anyone can open, as a plain **web API**, and as a **connector** that AI agents plug into directly. It has 34 automatic tests, a clean security scan, and real measured results.

2. The problem it solves

Companies and developers who use AI models run into three problems at once:

- **Cost**. AI is billed per token. Most prompts are full of filler words that cost money but add nothing. Over thousands of calls, this waste adds up.

- **Safety and compliance.** Prompts can accidentally leak private data (like a Social Security number), try to jailbreak the AI, ask it to write dangerous code, or request harmful content. There is often no gatekeeper stopping this.
- **Blindness.** Teams usually have no clear view of what is being spent, what is being sent, or whether anything risky is going through.

Distil solves all three in one place, so a team does not need three separate tools.

3. Background - the two source documents (explained simply)

The Observability Whitepaper was a design paper. Its main idea: for AI systems, you should measure everything by the **token** instead of by the request, and you should connect token usage to real-world costs like money, energy, and carbon. It also described a "prompt intelligence" layer that spots wasted words and recommends compression.

TokenOps was a shorter brief listing the features a real product should have: count tokens used and available, sort prompts by type, remember old prompts so they are not re-run, shrink prompts, run safety checks, block dangerous code and harmful content, alert on rule-breaking, and connect to different AI providers - plus notes on architecture, hosting, making it a connector, listing it on a marketplace, and controlling which code packages an AI agent is allowed to use.

Distil is **one single product** that does both. It does not exist as two separate things - every feature lives in the same code, the same live website, and the same connector.

4. How it works - step by step

When a prompt comes in, it flows through one pipeline:

Prompt comes in

```
|
v GOVERN      Sort the prompt by type. Scan for private data, secrets, and
|              trick attacks. Check for harmful content. Check for banned code
|              packages. Scan any code for dangerous patterns.
|              -> Decide: ALLOW, WARN, or BLOCK. (Blocked prompts stop here.)
v COMPRESS    Shrink the prompt. Free mode uses fast word-removal rules.
|              Paid mode uses an AI to rewrite it more cleanly.
v VERIFY      Optionally, ask an AI to score 0-100 whether the meaning was kept.
v MEASURE     Record tokens saved, money saved, carbon saved, and speed.
v METER       Count this against the user's plan and remaining quota.
|
v
```

Result: the verdict, the prompt type, the shorter prompt, and all the numbers.

All of this happens in a **single call**. The person or agent gets one answer back with everything in it.

An important design choice keeps costs safe: **anyone can use the free** word-removal mode with no cost, but the paid AI features only work with a secret key. **This is why the public demo is safe to show to anyone - random** visitors can try it, but they cannot spend the owner's AI budget.

5. What was built - the TokenOps features (in detail)

Each requirement from the TokenOps brief, and how it was built:

1. Count tokens used and available. The system counts the exact tokens in every prompt. It keeps a running total and compares it to a monthly budget you set, showing how many tokens you have used, how many are left, and how much your budget is stretched by compression. *Note:* the "available" number is measured against a budget you configure, not a live reading from the AI provider's billing system.

2. Sort prompts by type. The system reads each prompt and labels it as one of six types: **enquiry** (a question), **code**, **testing**, **draft** (writing an email or essay), **ppt** (a presentation), or **refinement** (improving text). It also gives a confidence level.

3. Remember old prompts, avoid re-running. The system keeps a memory of prompts it has already handled. If the same prompt (or a very similar one) comes in again, it returns the saved answer instantly instead of doing the work twice.

4. Shrink prompts to save tokens. This is the core feature. It offers four modes: a **free** rule-based mode (removes filler and low-value words), an **AI-quality** mode (an AI rewrites the prompt more cleanly), an **adaptive** mode (uses the free mode first and only calls the AI if the meaning drops), and a **safe** mode (checks the result and falls back to the original if the short version loses too much meaning). Measured savings average about 41%.

5. Safety checks on the content. The system scans each prompt for **private** data (**emails, phone numbers, Social Security numbers, credit cards**), **secrets** (API keys, passwords), and **trick attacks** ("ignore your instructions", jailbreak attempts). If it finds something serious, it blocks the prompt. *Note:* these checks use pattern-matching. They catch common cases well but are not a certified, audited security guarantee.

6. Block dangerous code. If a prompt contains code, the system scans it for risky patterns - things like running unchecked commands, unsafe data loading, SQL-injection risks, hard-coded passwords, and weak encryption - and can block

it. **Note:** it flags risky code for review; it does not write safe code for you or promise the code is safe.

7. Block harmful content. The system checks for violent, hateful, or self-harm content. When connected to an AI provider it uses that provider's professional moderation tool; otherwise it uses a basic word filter.

8. Alert on rule-breaking. Every time a prompt is blocked or flagged, the system records it as a "violation" that can be listed and reviewed on a dashboard. **Note:** it records violations; sending them to email or Slack automatically is planned but not yet built.

9. Connect to different AI providers. The system lists the AI providers it can work with (OpenAI, Anthropic/Claude, Google, Mistral, and local models) and shows which are set up. Because it works as a universal connector, any AI agent can plug into it. **Note:** today it actually calls OpenAI for the AI features; the others are documented as ready-to-connect.

Design - architecture. Fully documented, with a clean layered design.

Design - hosting. Live on Vercel's serverless platform, which costs nothing when idle. **Note:** long-term data storage needs one more free service (Upstash) to be connected.

Design - connector. Done. It is a live connector that AI agents can add, offering 20 tools.

Design - marketplace. The system publishes its own description file for a future marketplace listing, and there is a checklist of what a full listing needs. The formal listing itself is future work.

Design - code package policy. The system detects which software packages a piece of code wants to install or import, and checks them against an allowed/banned list you configure. It blocks prompts that try to pull banned packages.

In short: every feature in the TokenOps brief is addressed - most fully, and a few with honest notes about what is a first version versus a finished enterprise feature.

6. What was built - the source whitepaper (about 70%)

Built: measuring everything by tokens; connecting tokens to cost, energy, and carbon; the "prompt intelligence" layer (spotting waste and compressing); step-by-step timing of the pipeline; connecting token counts to cost and carbon savings; automatic detection of unusual patterns; smart optimization (shrink,

route to the right model, reuse cached answers); dashboards; and a forecast of future savings.

Handled honestly: the whitepaper wanted to measure real GPU hardware usage. Because the AI runs on the provider's servers (not ours), we cannot measure their GPUs. Instead the system **estimates** the compute load using a standard formula and measures our own server's CPU and memory, clearly labeling every estimate as an estimate.

Not built (needs special hardware or a live AI-serving setup): reading real GPU memory and cache usage, watching live AI traffic as it flows, and automatically scaling servers up and down. These are blocked by hardware and infrastructure, not by effort.

7. How to see it working (demo)

- **Easiest:** open <https://distil.vercel.app> and paste a prompt. Try a wordy question to see it shrink, then try a prompt with a fake Social Security number or an "ignore your instructions" line to watch it get **blocked**.
 - **Charts:** click "Engineering view" to see the live graphs.
 - **As an API:** send a prompt to the `/process` web address and get JSON back.
 - **As a connector:** add the connector link to Claude and let the AI use the tools itself.
-

8. Results and numbers

- **20 tools** available through the connector.
 - **34 automatic tests**, all passing.
 - **Security scan (Semgrep): 0 problems found.**
 - **Benchmark on 32 test prompts:** about **41% smaller prompts**, and the safety checks correctly caught **100% of the unsafe prompts with 0 false alarms on that test set.**
-

9. Honest limitations

These are stated plainly so nothing is oversold:

1. The free word-removal compressor is rule-based, so its output can read a little choppy. The AI-quality mode reads better but costs money.
2. The benchmark is our own small test set. It should be described as "100% on our 32-prompt test," not as proven general accuracy.

3. The safety checks use pattern-matching. They are good for common cases but are **not an independently audited, enterprise-grade guarantee**.
 4. Cost, energy, carbon, and GPU numbers are **estimates**, clearly labeled.
 5. Long-term data storage needs one more free service connected, and the billing is set up to measure usage but is not yet connected to real payments.
-

10. Future scope

This project is a strong, working foundation. There is a clear path to grow it from a working product into a real business. The future work falls into three layers.

10a. Near-term improvements (pure engineering)

- **Permanent memory.** Connect a free database (Upstash) so the metrics, usage history, and violation logs survive server restarts. This is the single most important next step for reliability.
- **Better compression quality.** Add a small trained model, or make the adaptive AI mode the default, so the shrunk prompts read as cleanly as the best competitors.
- **A customer dashboard.** A page where each customer logs in with their key and sees their own usage, plan, and blocked prompts - turning "an API" into "a product people log into."
- **Real alerts.** Send a message to email or Slack the moment a dangerous prompt is blocked.
- **More AI providers.** Actually call Anthropic, Google, and others, not just OpenAI, so customers are not locked to one vendor.
- **A bigger, tougher benchmark.** Test against larger, more adversarial data so the accuracy numbers hold up to scrutiny.

10b. Medium-term - turning it into a business

- **Payments.** Connect Stripe so the plan tiers (free / pro / enterprise) can actually charge customers. The usage-metering needed for this is already built.
- **Marketplace listing.** Complete the listing so businesses can discover and add the connector in a few clicks.
- **Self-serve sign-up.** Let a new customer create an account, get a key, and start using it without any manual steps.
- **Output checking.** Right now it checks prompts going in; it could also check the AI's answers coming out (for safety and for caching repeated answers).

10c. Long-term - the bigger vision

- **An "AI firewall" for companies.** As more businesses let AI agents write code and take actions, they will need a gatekeeper that enforces safety,

budget, and policy rules on every AI call. This product is a natural starting point for exactly that.

- **Compliance and reporting.** Companies in regulated industries (finance, healthcare) need proof of what their AI systems sent and blocked. The violation logs and metrics are the foundation for compliance reports.
- **Sustainability reporting.** As "green AI" becomes a real concern, the energy and carbon estimates could grow into proper sustainability dashboards for leadership.
- **An independent security audit.** To sell to large enterprises, the safety checks would need review and testing by professional security auditors. This cannot be done with code alone, but the product is structured to support it.

10d. Market potential

The market for **AI governance, observability, and cost control** is growing quickly as companies move AI from experiments into real production. Most tools today do only one piece - either compression, or safety, or monitoring. This product's advantage is that it does **all three in one gateway, with cost and carbon visibility built in. The realistic strategy is not to beat the biggest** compression tools at raw compression, but to win on "**safe, measured, all-in-one**" governance **for teams running AI agents.**

11. Conclusion

Distil is a complete, live, tested product that unifies the token-monitoring vision and the TokenOps feature brief into a single AI gateway. It genuinely shrinks prompts, blocks unsafe ones, measures cost and carbon, and works both as a website and as a connector for AI agents - all while keeping the owner's costs safe. It is honest about its limits, and it has a clear, realistic path to grow into a genuine business.

Prepared for demonstration. Every number is either measured or clearly labeled as an estimate. Nothing in this report is fabricated. The product is live and can be tried immediately at the links at the top.